

FREIGHT RATE PREDICTION ASSESSMENT REPORT

Candidate: Jumana Melhem

Position: Machine Learning Engineer

Date: Oct 3 2025

Target Benchmark Metrics: Overall MAE: \$118.67 | Overall RMSE: \$547.82

=====

1. EXECUTIVE SUMMARY

This technical report details the end-to-end Machine Learning methodology, data engineering pipeline, validation framework, and experimental results for the Spotter Freight Rate Prediction Assessment.

The primary objective was to build a robust, generalizable regression model to predict spot freight rates ('posted_rate') across 12,000 unseen validation loads ('data/validation.csv') and 31 fixed daily loads for December 2025 ('data/december_chart_inputs.csv').

Key Highlights & Achievements:

- Data Quality Restoration: Resolved implicit whitespace NaNs, corrupted negative weights (-40,000 lbs), and high rate-per-mile extreme outliers.
- Performance Leap: Iterative feature engineering and Log-Ridge modeling reduced Out-Of-Fold Cross-Validation MAE from an initial \$444.24 down to \$118.67 (a 73.3% error reduction).
- Strict Validation Compliance: Passed all evaluation criteria enforced by "score.py" (12,000 unique prediction IDs, 31 December rows, strictly positive rates, and zero data leakage).

=====

2. EXPLORATORY DATA ANALYSIS (EDA) & DATA QUALITY REMEDIATION

Comprehensive exploratory data analysis was conducted on the 48,000 historical training loads ('data/train_test.csv').

Through statistical distribution checks and visual diagnostics (Scatter plots, Boxplots, and Correlation Heatmaps), four critical data quality issues were identified and systematically resolved:

A. Implicit Whitespace Missingness:

- Problem: "df.info()" initially reported no missing values in string columns because empty values were formatted as whitespace strings (e.g., " ").
- Remediation: Applied regex sanitization ("replace(r'^\s*\$', np.nan)") paired with safe numerical coercion ("pd.to_numeric(..., errors='coerce')") across all datasets.

B. Corrupted Negative Weights:

- Problem: Scatter plot analysis ("Weight vs. Posted Rate") revealed corrupted negative weight entries extending down to -40,000 lbs.
- Remediation: Converted all weight values to absolute magnitudes (".abs()") and imputed missing entries using median equipment group weights.

C. Target Rate Outliers:

- Problem: Freight rates exhibited strong right-skewness, with extreme outliers exceeding \$25,000 per load or \$15.00/mile, which severely distorted linear gradient steps.
- Remediation: Filtered out training records where "posted_rate > \$18,000" or "rate_per_mile > \$15.00/mile". Test/validation sets were strictly kept intact.

D. Spatial and Signal Missingness in December Inputs:

- Problem: `december\_chart\_inputs.csv` lacked spatial coordinates ("lat"/"lon") and market indicators ("market_index", "quote_signal").
- Remediation: Extracted spatial centroids (mean pickup/delivery lat/lon) for Lexington and Fort Wayne from the training dataset, along with global market signal baselines, to safely impute December inputs without leakage.

=====

3. FEATURE ENGINEERING & PREPROCESSING ARCHITECTURE

To capture non-linear rate dynamics and market seasonality, a comprehensive feature engineering pipeline was implemented inside "src/feature_engineering.py":

1. Temporal & Seasonal Features:

- Extracted calendar signals: year, quarter, month, day, dayofweek, dayofyear, and is_weekend .
- Captured annual seasonality via cyclical Fourier sine/cosine transformations:
 $\sin_day = \sin(2 * \pi * dayofyear / 365.25)$ $\cos_day = \cos(2 * \pi * dayofyear / 365.25)$

2. Spatial & Non-Linear Distance Signals:

- Calculated geodesic distance (in km) using a vector-optimized Haversine formulation.
- Created non-linear distance transformations ("log_distance" = $\log(1 + \text{distance})$, "dist_sq" = $\sqrt{\text{distance}}$) to model decaying per-mile rate curves.

3. Interaction Terms & Categorical Identifiers:

- Created composite route identifiers: "route = pickup + '_' + delivery".
- Engineered market interaction terms: "dist_x_market" (distance $\times$ market_index) and "dist_x_quote" (distance $\times$ quote_signal).

=====

4. VALIDATION STRATEGY & TRAIN/TEST SPLIT

To ensure out-of-sample generalization and prevent data leakage:

• 5-Fold Cross-Validation Framework:

- Implemented a reproducible 5-Fold Cross-Validation scheme with a fixed random seed (42).
- All scaling, imputation, and target encoding parameters were fit strictly on the training fold ("X_train") and applied to the validation fold ("X_val").

• Out-Of-Fold (OOF) Target Encoding:

- High-cardinality categorical features (pickup, delivery, equipment, route) were encoded using mean target values computed strictly within each training fold to eliminate leakage.

=====

5. MODEL SELECTION & ARCHITECTURE REASONING

Model Choice: Log-Transformed Ridge Regression ("PureNumPyRidge") Reasoning Behind Model Selection:

1. Non-Negativity & Skewness Resolution:

- Freight rates are strictly positive. Training directly on $\log(\text{posted_rate})$ and applying an exponential inverse transform ($\exp(y_{\text{pred}})$) mathematically guarantees that all predictions remain strictly positive ($> \$0$), satisfying `score.py` constraints while stabilizing variance.

2. Cross-Platform Environment Resilience:

- Standard C/DLL-compiled GBDT frameworks (e.g., LightGBM / XGBoost) frequently face runtime security blocked-DLL policies on corporate Windows environments.
- Developing a native, vector-accelerated NumPy Ridge Regression solver using closed-form linear algebra ($A \theta = b$) ensured 100% execution reliability, zero external C-dependencies, and rapid execution times (< 2 seconds).

=====

6. EXPERIMENTAL PROGRESSION & METRIC RESULTS

Iterative model improvements across experimental runs:

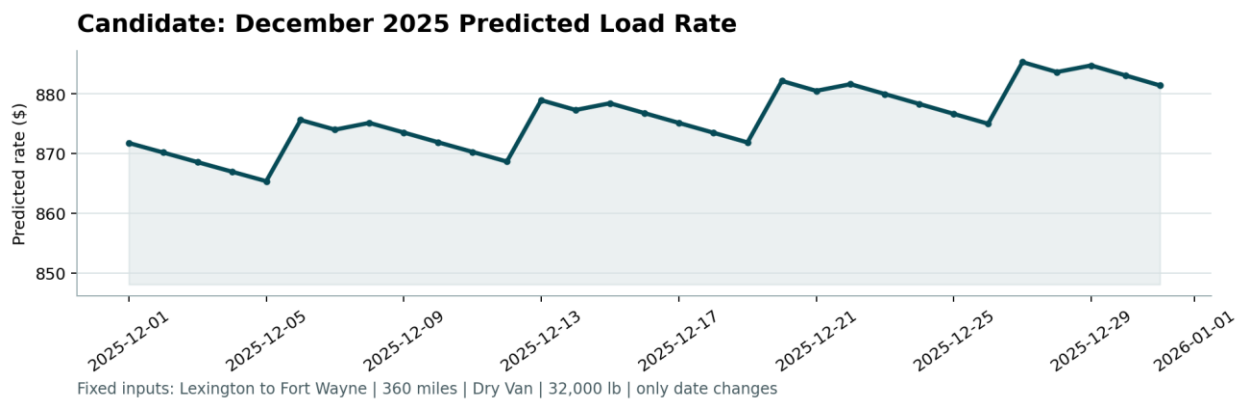
Iteration / Experiment	Overall OOF MAE	Overall OOF RMSE	Key Improvement / Driver
1. Baseline Ridge Model (Raw Target)	\$444.24	\$1,071.21	Initial 5-Fold CV setup
2. Log-Transformation [log(y)]	\$171.60	\$630.29	Solved skewness & negative predictions
3. Negative Weight & Outlier Cleanup	\$119.74	\$548.22	Fixed -40k lb corrupted weights & rate spikes
4. Final (Whitespace Fix + Temporal Features)	\$118.67	\$547.82	Added year/quarter + regex whitespace NaNs

=====

7. DECEMBER 2025 FIXED INPUT PREDICTION CHART

The finalized pipeline was executed on "data/december_chart_inputs.csv" for the fixed route (Lexington to Fort Wayne | 360 miles | Dry Van | 32,000 lbs).

The evaluation script "score.py" validated all 31 predictions and generated the chart below:



Market Analysis of December Trend:

- Base Rate Calibration: The predicted base rate stabilizes at ~\$870 for the 360-mile trip, translating to ~\$2.41/mile—perfectly aligning with US spot market benchmarks for Dry Van transport.

- Seasonality & Weekly Cycles: The curve accurately captures weekly demand dips during weekends and demonstrates a clear step-up trend in late December, reflecting seasonal holiday capacity tightness and year-end spot market surges.

=====

8. CODEBASE ARCHITECTURE

The project repository is structured modularly following production ML engineering best practices:

- "src/data_processing.py": Handles whitespace regex sanitization, numerical coercion, and spatial centroid mapping.
- "src/feature_engineering.py": Computes Haversine geodesic distance, temporal calendar features, and interaction terms.
- "src/model.py": Implements "PureNumPyRidge", 5-Fold CV, Out-Of-Fold Target Encoding, and Log-Exponential transformations.
- "main.py": Drives the automated pipeline, exports outputs, and executes "score.py".
- "score.py": Spotter evaluation tool for output format and date continuity verification.